MODULE 6 OF 7

HTTPS, HEADERS & TRANSPORT SECURITY

Transport Security for AI Directed Engineers

TIER 6 — THE VAULT • SECURITY FOUNDATIONS

MATT MURPHY .AI

mattmurphy.ai

Part of the AI Directed Engineering Certification Series

What This Module Covers

This module covers transport security—how data moves between your user's browser and your server, and the HTTP security headers that instruct browsers to protect your users from common attacks. SSL/TLS configuration, HTTPS enforcement, security headers like Content-Security-Policy, Strict-Transport-Security, X-Frame-Options, and X-Content-Type-Options form a protective layer between your application and the internet. Transport security is the layer that most AI tools completely ignore. Your AI builds the application. Your hosting platform usually provides a basic SSL certificate. But the security headers that prevent XSS escalation, clickjacking, MIME sniffing, and protocol downgrade attacks are almost never added by default. They require explicit configuration—and they are among the easiest and highest-impact security improvements you can make.

Why It Matters

Without HTTPS, every piece of data between your user and your server travels in plain text. Login credentials, session tokens, personal information—all readable by anyone on the same network. Public WiFi at a coffee shop becomes a surveillance tool. Without security headers, your application trusts the browser to make security decisions with no guidance. A browser will happily load your page in an attacker's iframe, execute inline scripts injected by XSS, or interpret a text file as JavaScript. Security headers are instructions to the browser: don't allow these dangerous behaviors.

MODULE CERTIFICATION GOAL

You can evaluate and configure transport security for AI-built applications—HTTPS enforcement, TLS configuration, and security headers including CSP, HSTS, X-Frame-Options, and X-Content-Type-Options—and verify that your application's browser-to-server communication is protected against interception and manipulation.

What You Need to Know

HTTPS and TLS: HTTPS encrypts all traffic between browser and server using TLS (Transport Layer Security). Without it, data travels in plain text. Modern browsers mark HTTP sites as 'Not Secure.' TLS certificates are free through Let's Encrypt and most hosting platforms provision them automatically. But automatic provisioning doesn't mean automatic enforcement—your app must redirect all HTTP requests to HTTPS.

Content-Security-Policy (CSP): CSP tells the browser which sources of content are allowed on your page. A strict CSP prevents inline scripts from executing, blocks loading resources from unauthorized domains, and significantly reduces the impact of XSS vulnerabilities. Even if an attacker injects a script tag, CSP can prevent it from executing.

Strict-Transport-Security (HSTS): HSTS tells the browser to always use HTTPS for your domain, even if the user types http://. Without HSTS, the first request might go over HTTP before redirecting—a window where an attacker can intercept the connection. HSTS closes that window.

X-Frame-Options and frame-ancestors: These headers prevent your application from being loaded inside an iframe on another site. Without them, an attacker can overlay your app inside their page and trick users into clicking your buttons while thinking they're on the attacker's site. Set to DENY or SAMEORIGIN unless you specifically need iframe embedding.

X-Content-Type-Options: This header prevents browsers from MIME-sniffing—guessing the content type of a response and potentially treating a text file as executable JavaScript. Set to 'nosniff' to ensure browsers respect the Content-Type header you explicitly set.

Your Toolkit

AI coding tool (pick one): Cursor, Lovable, Bolt, Claude Code—direct it to add security headers to your application's response middleware or server configuration.

Security header scanner: SecurityHeaders.com—enter your URL and get an instant grade on your security header configuration with specific recommendations.

SSL/TLS testing: SSL Labs Server Test (ssllabs.com/ssltest)—evaluates your TLS configuration and certificate chain for weaknesses.

CSP evaluator: CSP Evaluator (csp-evaluator.withgoogle.com)—checks your Content-Security-Policy header for common misconfigurations and bypasses.

Certification Exam Topics

Every exam question is scenario-based. You'll see a situation and need to identify what's right, what's wrong, or what to do next. Here's what gets tested:

HTTPS enforcement: Can you verify that all HTTP requests are redirected to HTTPS and no application content is served over unencrypted connections?

CSP configuration: Can you evaluate whether a Content-Security-Policy header effectively restricts script sources and prevents inline script execution?

HSTS implementation: Can you assess whether HSTS is configured to prevent protocol downgrade attacks from HTTPS to HTTP?

Clickjacking prevention: Can you verify that X-Frame-Options or CSP frame-ancestors headers prevent your application from being loaded in unauthorized iframes?

MIME sniffing prevention: Can you check whether X-Content-Type-Options is set to prevent browsers from interpreting files as a different content type?

Certificate management: Can you evaluate whether TLS certificates are valid, properly chained, and set to auto-renew before expiration?

Mixed content: Can you identify when an HTTPS page loads resources over HTTP which browsers block or warn about as mixed content?

Referrer policy: Can you assess whether the Referrer-Policy header is configured to prevent your URLs from leaking sensitive information to third-party sites?

Common Pitfalls

These are the mistakes vibecoders make most often in this area. No judgment—they're easy to make. But if you recognize any of them in your own workflow, fix them before sitting for the exam.

Having an SSL certificate but not redirecting HTTP to HTTPS. If your app responds on both protocols, users who type your URL without https:// get the unencrypted version. All HTTP requests must redirect to HTTPS.

Not setting a Content-Security-Policy header because the app works without it. CSP doesn't add features—it restricts what the browser is allowed to do. Without it, an XSS vulnerability has maximum impact.

Setting CSP to 'unsafe-inline' because your AI generated inline scripts. This effectively disables CSP's XSS protection. Refactor inline scripts to external files so you can use a strict CSP.

Forgetting HSTS on the first deploy and adding it later. HSTS only protects users who have visited your site after the header was added. New users on their first visit are still vulnerable to downgrade attacks until they receive the header.

Not checking for mixed content. Your page loads over HTTPS but includes an image, font, or script over HTTP. The browser blocks or warns about it, and the page may display incorrectly or fail to load resources.

Letting SSL certificates expire. Even with auto-renewal configured, verify it actually works. An expired certificate shows a full-page browser warning that makes your site look compromised to every visitor.

Self-Assessment Checklist

Before you take the exam, run through these questions. Every "no" is something to work on.

Does every HTTP request to your application redirect to HTTPS?

Is a Content-Security-Policy header set that restricts script sources and blocks inline scripts?

Is Strict-Transport-Security (HSTS) configured with an appropriate max-age?

Is X-Frame-Options or CSP frame-ancestors set to prevent clickjacking?

Is X-Content-Type-Options set to 'nosniff'?

Are all resources on your HTTPS pages also loaded over HTTPS (no mixed content)?

Is your TLS certificate valid, properly chained, and set to auto-renew?

AI Audit Prompt Template

Copy this prompt into your AI coding tool to get a quick health check. It checks the same things the certification exam covers.

> Review my transport security configuration and check the following. For each one, tell me pass or fail with a specific example: HTTPS: Are all HTTP requests redirected to HTTPS? CSP: Is a Content-Security-Policy header set that blocks unsafe inline scripts? HSTS: Is Strict-Transport-Security configured? Clickjacking: Is X-Frame-Options or CSP frame-ancestors set? MIME sniffing: Is X-Content-Type-Options set to nosniff? Mixed content: Are all page resources loaded over HTTPS? Give me an overall score out of 6 and list the top 3 things to fix first.

What's Next

Once you can answer yes to the self-assessment checklist, you're ready for the Module 6 exam. The best way to prepare: visit securityheaders.com and scan your own application. Run the SSL Labs test. Check your browser's console for mixed content warnings. Every missing header and misconfiguration is exactly what the exam tests.

CERTIFICATION PATHWAY

Security Specialist — Foundations: Pass all 7 module exams in this course

CADE Specialist (meta-credential): CADE Certified + 3 specialist badges

CADE Distinguished: CADE Certified + 6 specialist badges

Each exam requires 80% to pass. You can retake after a 24-hour cooldown. No rush—take the time to build something real first.

MATT MURPHY .AI

mattmurphy.ai